

Daily Paper Review: FreeToken: Efficient Edge-Native MoE Serving with Bandwidth-Adaptive Execution

Auto-generated Report

August 20, 2026

1 Paper Summary

1.1 Problem and Motivation

Frontier open-weight Mixture-of-Experts (MoE) models such as DeepSeek-V4-Flash (284B) and GLM-5.2 (753B) approach proprietary-model capability, yet serving them remains confined to datacenter GPU clusters costing millions of dollars. Over 100 million consumer machines already contain discrete GPUs—gaming desktops, workstations, high-end laptops—representing an enormous pool of underutilized compute. The missing piece is a serving system that treats each heterogeneous consumer machine as a unified inference platform.

MoE architectures are both an opportunity and a challenge for edge deployment. DeepSeek-V4-Flash activates only 6 of 256 routed experts per token (13B of 284B parameters), drastically reducing per-token computation—but **sparsity reduces FLOPs without proportionally reducing memory**, since the complete expert pool must remain accessible. Yang et al. (arXiv:2608.16157) identify three core challenges: (1) *prefill-stage transfer costs*—an FP4 deployment of DeepSeek-V4-Flash requires transferring ~ 140 GB of expert weights, taking 2–10+ seconds depending on PCIe generation; (2) *decode-stage cache misses*—static expert placement misses the majority of routed traffic, and consumer CPU bandwidth (~ 50 – 90 GB/s) cannot sustain decode alone; (3) *resource management*—VRAM budgets fluctuate on shared consumer systems, and engine startup involves 20+ seconds of disk loading.

1.2 Method

FreeToken implements a **two-level expert-memory hierarchy**: a CPU-resident expert pool in pinned host DRAM (authoritative, always complete) and an elastic GPU expert cache in VRAM (LRU-managed subset for fast access). The GPU cache affects only performance, never correctness—exact MoE computation is preserved without any algorithmic approximation.

Bandwidth-Adaptive Execution (the q^* policy). The central contribution is a closed-form formula for splitting expert cache misses between PCIe transfer and CPU execution. Given m cache misses, each expert of size S bytes, measured pinned-transfer bandwidth B_P and host-processing bandwidth B_H , the optimal partition is:

$$q^* = m \cdot \frac{B_P}{B_H} \quad (1)$$

where q^* experts are transferred via PCIe to GPU (the *cache-fill set*) and $m - q^*$ are executed directly on CPU from the host pool (the *CPU-execution set*). Both branches execute concurrently. The derivation equalizes their execution times $T_{\text{fill}}(q) = qS/B_P$ and $T_{\text{cpu}}(m - q) = (m - q)S/B_R$ where $B_R = \max(B_H - B_P, 0)$. This single formula automatically adapts to any hardware: on an RTX 5090 ($B_P=52.7$, $B_H=77.3$), most misses go to PCIe fill; on an RTX 4060 laptop ($B_P=11.8$, $B_H=47.5$), most go to CPU execution.

Full-Layer Double-Buffered Prefill. Two full-layer buffers are allocated from the GPU slot pool. While the GPU computes routed experts of layer l from one buffer, a transfer stream loads the complete expert set of layer $l+1$ into the other—before that layer’s routing is even known. This hides expert transfer behind computation, achieving 6.7k tok/s at 16k-token prefills.

Semantic-Aware State Caching. A small pool of recurrent-state checkpoints is maintained at *semantic anchors*—special-token boundaries marking thinking segments, tool calls, and conversation turns. When agentic workloads modify context (stripping thinking blocks, replacing tool outputs), FreeToken restores from the deepest surviving checkpoint rather than re-prefilling the entire context.

CUDA-Graph-Compatible LRU Cache. All routing-dependent control resides on the GPU within statically captured CUDA graphs, avoiding host synchronization. A single GPU kernel performs expert deduplication, residency classification, q^* derivation, single-pass victim selection, and logical-to-physical ID rewriting.

Elastic Memory Management. After non-expert weights are allocated, remaining VRAM is divided between KV cache pages and expert slots. This division can be rebuilt at any scheduler safe point without restarting the engine or reloading the CPU expert pool.

1.3 Results

Experiments span six hardware systems (RTX 3090 through RTX PRO 6000), three models (Qwen3.6-35B-A3B, DeepSeek-V4-Flash 284B, GLM-5.2 753B), and four workloads (math reasoning, coding agent, native coding agent, email/calendar agent).

On an RTX 5090, FreeToken achieves 77–83 tok/s on Qwen3.6 (1.8–2.3 $\times$ over the strongest baseline) and 22–25 tok/s on DeepSeek-V4-Flash (1.5–1.9 $\times$). Decode throughput varies within only 12% across single-turn and agentic workloads, whereas KTransformers loses 31% under agentic context edits. Time-to-first-token stays below 44 s in all settings; baselines peak at 179–946 s.

Cross-hardware scaling is remarkably robust. An RTX 4060 *laptop* with 8 GB VRAM serves a 35B model at 39.3 tok/s—92% of the RTX 4090 rate and exceeding Codex’s reported 33 tok/s median. An RTX 5090 desktop loses only 4% versus the server configuration, while llama.cpp loses 20%. At frontier scale, a single RTX PRO 6000 (96 GB) serves the 753B GLM-5.2 at 14.9 tok/s (2.0 $\times$ over llama.cpp).

FreeToken’s LRU cache achieves a 16% miss rate on Qwen3.6 versus 41% for KTransformers’ prefill-updated placement and 62% for llama.cpp’s static split. Full-layer double buffering eliminates 19–26% throughput losses observed without pipelining.

1.4 Limitations

The authors acknowledge several constraints. (1) Multi-second prefill latency persists on consumer hardware, especially on PCIe 4.0 x8 laptop links (10+ seconds for 140 GB transfers). (2) LRU miss rates remain non-trivial for models with large expert pools (39% for DeepSeek-V4-Flash with 11% of the pool cached). (3) Consumer CPU bandwidth caps at $\sim$ 50 GB/s for dual-channel DDR4, bounding the CPU execution branch. (4) The system targets single-user edge deployment; multi-user concurrent serving is not addressed. (5) The q^* formula assumes stable measured bandwidths, which may fluctuate under system load.

1.5 Relevance to Research

FreeToken directly addresses LLM inference optimization (Topic T1) at the system level. The bandwidth-adaptive execution paradigm connects to our prior reviews of ELDR (Review #83, expert-locality-aware decode routing for disaggregated MoE serving), which optimizes expert placement in datacenter settings—FreeToken solves the analogous problem for edge devices. The elastic memory management relates to IndexCache (Review #9) and LookaheadKV (Review #10), which optimize KV cache allocation; FreeToken extends this to joint KV-expert cache budgeting. The semantic-aware checkpointing for agentic workloads connects to the agent memory challenges studied in Memanto (Review #40) and EvoArena (Review #71).

2 Follow-up Research Directions

2.1 Direction 1: Bandwidth-Adaptive Expert Routing for Diffusion LLM Inference

Gap: FreeToken’s q^* policy is designed for autoregressive MoE decoding where tokens are generated sequentially. Diffusion LLMs generate all tokens in parallel through iterative denoising, creating fundamentally different expert access patterns—each denoising step activates experts for *all* positions simultaneously, producing massive batched expert requests rather than sequential single-token queries.

Approach: Extend the bandwidth-adaptive framework to batched expert requests in diffusion LLM inference. The q^* formula should generalize to $q^* = m_{\text{batch}} \cdot B_P/B_H$ where m_{batch} accounts for expert deduplication across the full-sequence denoising batch. Diffusion models may exhibit higher expert reuse across positions within a single denoising step, potentially improving cache hit rates. Combine with the parallel decoding strategies from DMax (Review #29) and the diffusion-specific optimizations from S2D2 (Review #19).

Feasibility: Medium-high. The core q^* formula is architecture-agnostic; the main engineering challenge is adapting the CUDA-graph-compatible LRU cache to handle batched expert requests efficiently without exceeding VRAM budgets.

2.2 Direction 2: Predictive Expert Prefetching via Continual Routing Pattern Learning

Gap: FreeToken’s LRU cache is reactive—it relies on temporal locality but does not predict future expert demands. The 39% miss rate for DeepSeek-V4-Flash suggests significant room for improvement. Prior predictive approaches (MoE-Infinity, ProMoE) use static prediction models that cannot adapt to evolving user workloads or new model deployments.

Approach: Add a lightweight continual learning module that tracks routing patterns across sessions and adapts its expert prefetch predictions over time. Use a small recurrent model or learned hash table that maps recent routing history to next-step expert predictions, trained incrementally via online learning without catastrophic forgetting (connecting to the EWC and LoRA-based continual learning methods from Macaron-V1, Review #111). The predictor would operate alongside the LRU cache, prefetching high-confidence experts during idle GPU cycles while falling back to the q^* policy for misses.

Feasibility: Medium. The predictor must be extremely lightweight (microsecond inference) to avoid adding latency to the decode path. The continual learning component must be stable across diverse workloads without periodic retraining.

2.3 Direction 3: Federated Edge MoE Serving with Expert Specialization

Gap: FreeToken treats each edge device as an isolated inference platform. In practice, many edge environments (research labs, offices, gaming communities) have multiple machines on a

local network. Distributing expert subsets across machines could enable serving models that exceed any single machine’s memory, but naive distribution ignores routing locality and network latency.

Approach: Extend FreeToken’s two-level hierarchy to a three-level system: GPU cache $\rightarrow$ local CPU pool $\rightarrow$ LAN-resident expert shards on peer machines. Use routing statistics to assign experts to machines based on co-activation patterns (experts frequently activated together should reside on the same node). The q^* formula generalizes to three bandwidth tiers: B_P (PCIe), B_H (local CPU), and B_N (network). This connects to the disaggregated serving paradigm of ELDR (Review #83) but targets consumer-grade LAN environments rather than datacenter fabrics.

Feasibility: Medium. LAN bandwidth ($\sim 1\text{--}12.5$ GB/s for 10–100 GbE) is an order of magnitude below PCIe, so expert placement optimization becomes critical. The main risk is that network latency variance may make the static bandwidth model unreliable.

3 Conclusion

FreeToken demonstrates that frontier MoE models can be served interactively on consumer hardware through careful systems engineering. The bandwidth-adaptive q^* policy—a single closed-form formula that optimally splits expert cache misses between PCIe transfer and CPU execution—is both elegant and practically effective, delivering $1.5\text{--}2.3\times$ throughput improvements across six hardware configurations. The result that an 8 GB laptop GPU can serve a 35B model at 39.3 tok/s, or a single workstation GPU can run the 753B GLM-5.2, challenges the assumption that frontier model inference requires datacenter infrastructure. Combined with semantic-aware caching for agentic workloads and elastic memory management, FreeToken provides a comprehensive serving stack that makes model accessibility a function of hardware ownership rather than cloud budget. As MoE architectures continue to scale, bandwidth-adaptive execution will be essential infrastructure for democratizing access to frontier AI.

References

- [1] S. Yang, X. Fan, M. Pan, H. Xi, Z. Wang, S. Sun, K. Keutzer, S. Han, M. Zaharia, C. Xu, and I. Stoica. FreeToken: Efficient Edge-Native MoE Serving with Bandwidth-Adaptive Execution. *arXiv preprint arXiv:2608.16157*, 2026.
- [2] DeepSeek-AI. DeepSeek-V4-Flash Technical Report. *arXiv preprint*, 2026.
- [3] Y. Zhang et al. KTransformers: A Flexible Framework for Experiencing Cutting-edge LLM Inference Optimizations. *arXiv preprint arXiv:2411.00024*, 2024.
- [4] L. Xue et al. MoE-Infinity: Offloading-Efficient MoE Model Serving. *arXiv preprint arXiv:2401.14361*, 2024.
- [5] Y. Song et al. PowerInfer: Fast Large Language Model Serving with a Consumer-grade GPU. *arXiv preprint arXiv:2312.12456*, 2023.